



Instituto Politecnico
Nacional



Unidad profesional
interdisciplinaria de ingeniería,
ciencias sociales y administrativa

Ingeniería en informática
Administracion de Base de Datos
4NV40

Base de Datos: “Tiendita”

Integrantes:

Lopez Ramos Leonardo Arturo

Magaña Soriano Marco Antonio

Huerta Alonso Andrea

Contenido

Modelado del Negocio	3
1. MODELO CONTEXTUAL	3
2. MODELO ENTIDAD-RELACIÓN (E-R)	5
3. Modelo Relacional	7
4. Modelo Fisico/Logico	8
5. Cconsultas del administrador	12

Modelado del Negocio

1. MODELO CONTEXTUAL

¿Qué es?

El modelo contextual es el **nivel más alto de abstracción** de una base de datos. Muestra el sistema en su entorno, identificando **qué entidades externas interactúan con él** y **qué flujos de información** existen, sin importar cómo está estructurada la base de datos internamente.

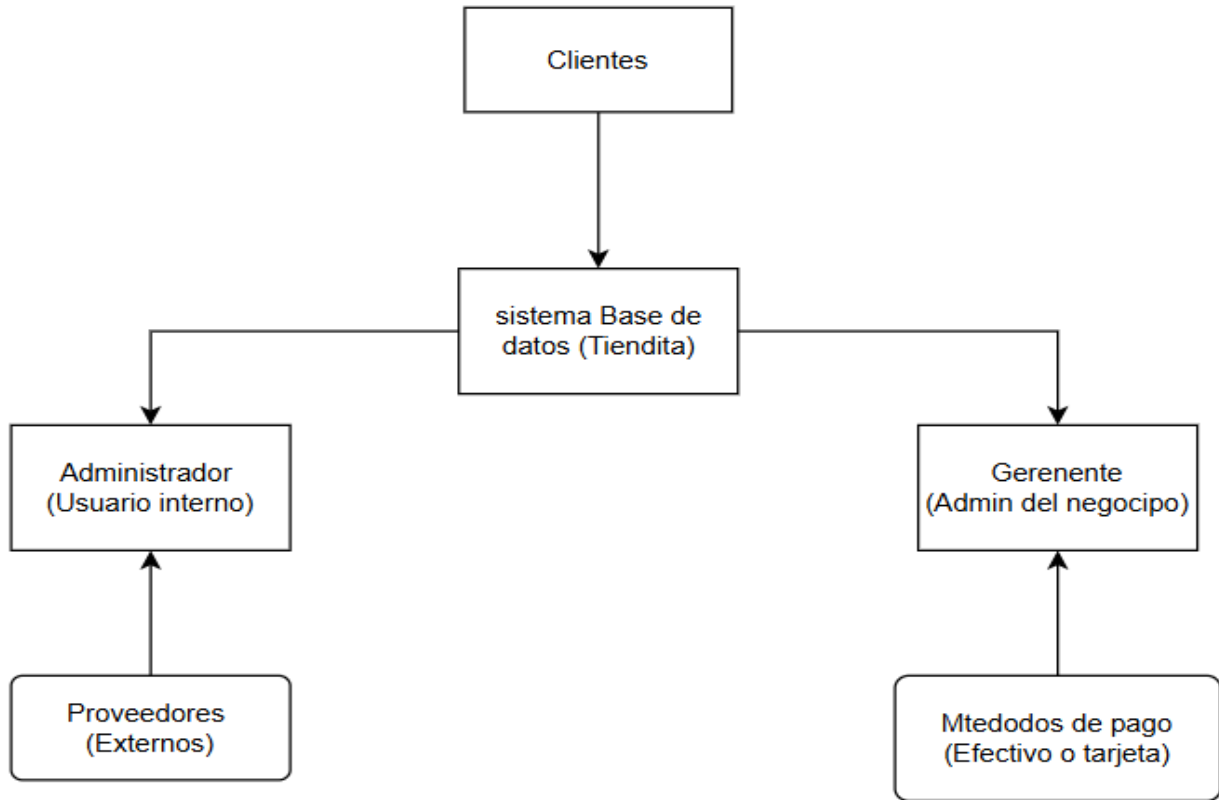
Es como ver la tienda desde afuera: sabemos que entran clientes, que llegan productos de proveedores, que hay un administrador, pero no vemos los estantes ni la caja registradora.

Entidad Externa	Descripción	Interacción con el Sistema
Clientes	Personas que compran en la tienda	Generan las ventas al realizar compras
Administrador	Encargado de gestionar la tienda	Da de alta productos, actualiza precios, gestiona proveedores
Gerente / Administración	Toma decisiones estratégicas	Consulta reportes de ventas, analiza rendimiento
Proveedores	Empresas que suministran productos	Proveen los productos que se venden (información externa)
Métodos de Pago	Sistemas bancarios o de pago	Procesan las transacciones (efectivo, tarjeta, transferencia)

Flujos de información principales:

- Los **clientes** generan ventas → el sistema registra tickets
- El **administrador** alimenta el sistema con productos y precios
- El **gerente** extrae reportes del sistema para tomar decisiones
- Los **proveedores** existen como datos, pero su interacción es externa

Muestra que tu base de datos no existe en el vacío, sino que sirve a actores reales con necesidades reales.

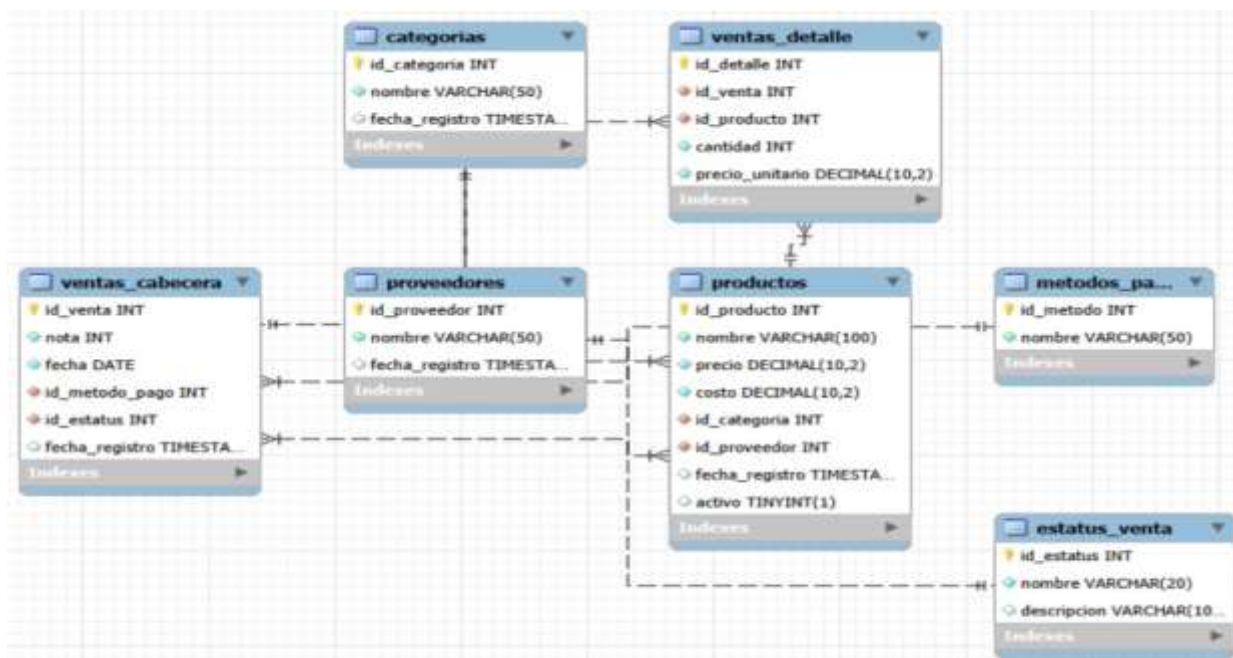


2. MODELO ENTIDAD-RELACIÓN (E-R)

Es una representación conceptual de los datos. Muestra las entidades (objetos del mundo real) que existen en el negocio, sus atributos (características) y las relaciones entre ellas. Es como el plano arquitectónico de tu base de datos: muestra qué "cosas" existen y cómo se conectan, pero todavía no cómo se construyen técnicamente.

Entidad Externa	Descripción	Interacción con el Sistema
Cientes	Personas que compran en la tienda	Generan las ventas al realizar compras
Administrador	Encargado de gestionar la tienda	Da de alta productos, actualiza precios, gestiona proveedores
Gerente / Administración	Toma decisiones estratégicas	Consulta reportes de ventas, analiza rendimiento
Proveedores	Empresas que suministran productos	Proveen los productos que se venden (información externa)
Métodos de Pago	Sistemas bancarios o de pago	Procesan las transacciones (efectivo, tarjeta, transferencia)

Separaste Proveedores de Productos (un proveedor puede tener muchos productos), separaste Cabecera de Detalle (un ticket puede tener múltiples productos), y creaste catálogos independientes para Categorías, Métodos de Pago y Estatus.



Relación	Tipo	Significado en el negocio
Proveedores → Productos	1:N	Un proveedor puede tener muchos productos, pero cada producto tiene UN proveedor
Categorías → Productos	1:N	Una categoría agrupa muchos productos, cada producto pertenece a UNA categoría
Productos → Ventas_Detalle	1:N	Un producto puede venderse muchas veces, en muchos tickets diferentes
Ventas_Cabecera → Ventas_Detalle	1:N	Una venta (ticket) puede tener muchos productos (detalles)
Metodos_Pago → Ventas_Cabecera	1:N	Un método de pago puede usarse en muchas ventas
Estatus_Venta → Ventas_Cabecera	1:N	Un estatus puede aplicarse a muchas ventas

3. Modelo Relacional

El modelo relacional es la traducción del modelo E-R a tablas concretas. Especifica llaves primarias (PK), llaves foráneas (FK) y las conexiones exactas entre tablas. Es el puente entre el "qué" (modelo E-R) y el "cómo" (modelo físico).

Tabla	Llave Primaria (PK)	Llaves Foráneas (FK)
PROVEEDORES	id_proveedor	Ninguna
CATEGORIAS	id_categoria	Ninguna
METODOS_PAGO	id_metodo	Ninguna
ESTATUS_VENTA	id_estatus	Ninguna

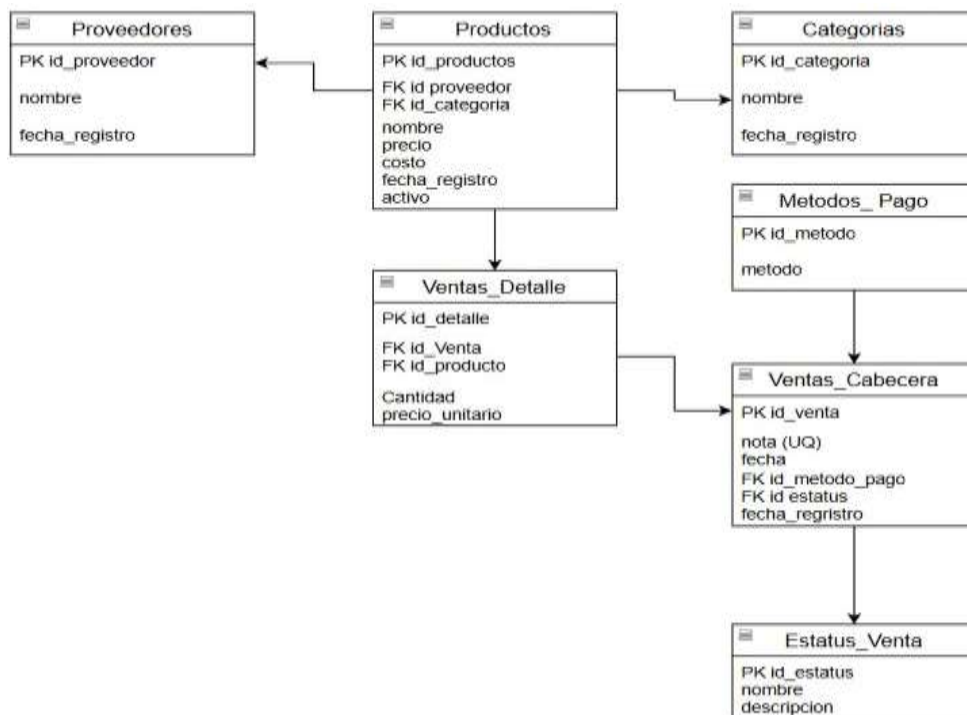
Reglas de integridad relacional:

Integridad de entidad: Cada tabla tiene una llave primaria única y no nula

Integridad referencial: No puede existir un producto con un proveedor que no exista

Integridad de dominio: Cada columna solo acepta valores de su tipo definido

Las llaves foráneas garantizan que los datos sean consistentes: no puedes tener una venta de un producto que no existe, ni un producto con una categoría eliminada.



4. Modelo Fisico/Logico

El modelo físico es la implementación real en un motor de base de datos específico (en tu caso, MySQL 8.0+). Incluye tipos de datos concretos, índices, restricciones, motores de almacenamiento y todas las decisiones técnicas que afectan el rendimiento y la integridad.

Tipo de Dato	Uso en Tiendita	¿Por qué?
INT	Llaves primarias, cantidades, notas	Números enteros, eficientes para búsquedas y joins
VARCHAR(50/100)	Nombres de productos, categorías, proveedores	Texto variable, ahorra espacio
DECIMAL(10,2)	Precios, costos, subtotales	Precisión exacta para dinero (evita errores de redondeo)
DATE	Fecha de venta	Solo fecha, sin hora (optimizado)
TIMESTAMP	Fecha de registro	Automático, con zona horaria
BOOLEAN	activo en Productos	Verdadero/Falso (TINYINT internamente)

```
-- MySQL Workbench Forward Engineering
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
-- Schema mydb
-- Schema tiendita
DROP SCHEMA IF EXISTS tiendita ;
-- Schema tiendita
CREATE SCHEMA IF NOT EXISTS tiendita DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci ;
USE tiendita ;
-- Table tiendita.categorias
```

```
DROP TABLE IF EXISTS tiendita.categorias ;
```

```
CREATE TABLE IF NOT EXISTS tiendita.categorias (  
  id_categoria INT NOT NULL AUTO_INCREMENT,  
  nombre VARCHAR(50) NOT NULL,  
  fecha_registro TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id_categoria),  
  UNIQUE INDEX nombre (nombre ASC) VISIBLE)  
ENGINE = InnoDB  
AUTO_INCREMENT = 11  
DEFAULT CHARACTER SET = utf8mb4  
COLLATE = utf8mb4_0900_ai_ci;  
-- Table tiendita.estatus_venta  
DROP TABLE IF EXISTS tiendita.estatus_venta ;
```

```
CREATE TABLE IF NOT EXISTS tiendita.estatus_venta (  
  id_estatus INT NOT NULL AUTO_INCREMENT,  
  nombre VARCHAR(20) NOT NULL,  
  descripcion VARCHAR(100) NULL DEFAULT NULL,  
  PRIMARY KEY (id_estatus),  
  UNIQUE INDEX nombre (nombre ASC) VISIBLE)  
ENGINE = InnoDB  
AUTO_INCREMENT = 4  
DEFAULT CHARACTER SET = utf8mb4  
COLLATE = utf8mb4_0900_ai_ci;
```

```
-- Table tiendita.metodos_pago  
DROP TABLE IF EXISTS tiendita.metodos_pago ;  
CREATE TABLE IF NOT EXISTS tiendita.metodos_pago (  
  id_metodo INT NOT NULL AUTO_INCREMENT,  
  nombre VARCHAR(50) NOT NULL,  
  PRIMARY KEY (id_metodo),  
  UNIQUE INDEX nombre (nombre ASC) VISIBLE)  
ENGINE = InnoDB  
AUTO_INCREMENT = 5  
DEFAULT CHARACTER SET = utf8mb4  
COLLATE = utf8mb4_0900_ai_ci;
```

```
-- Table tiendita.proveedores
DROP TABLE IF EXISTS tiendita.proveedores ;
CREATE TABLE IF NOT EXISTS tiendita.proveedores (
  id_proveedor INT NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  fecha_registro TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_proveedor),
  UNIQUE INDEX nombre (nombre ASC) VISIBLE)
ENGINE = InnoDB
AUTO_INCREMENT = 11
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;
```

```
-- Table tiendita.productos
DROP TABLE IF EXISTS tiendita.productos ;
CREATE TABLE IF NOT EXISTS tiendita.productos (
  id_producto INT NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(10,2) NOT NULL,
  costo DECIMAL(10,2) NOT NULL,
  id_categoria INT NOT NULL,
  id_proveedor INT NOT NULL,
  fecha_registro TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  activo TINYINT(1) NULL DEFAULT '1',
  PRIMARY KEY (id_producto),
  INDEX id_categoria (id_categoria ASC) VISIBLE,
  INDEX id_proveedor (id_proveedor ASC) VISIBLE,
  CONSTRAINT productos_ibfk_1
    FOREIGN KEY (id_categoria)
      REFERENCES tiendita.categorias (id_categoria),
  CONSTRAINT productos_ibfk_2
    FOREIGN KEY (id_proveedor)
      REFERENCES tiendita.proveedores (id_proveedor))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;
```

```
-- Table tiendita.ventas_cabecera
DROP TABLE IF EXISTS tiendita.ventas_cabecera ;
```

```

CREATE TABLE IF NOT EXISTS tiendita.ventas_cabecera (
  id_venta INT NOT NULL AUTO_INCREMENT,
  nota INT NOT NULL,
  fecha DATE NOT NULL,
  id_metodo_pago INT NOT NULL,
  id_estatus INT NOT NULL,
  fecha_registro TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_venta),
  UNIQUE INDEX nota (nota ASC) VISIBLE,
  INDEX id_metodo_pago (id_metodo_pago ASC) VISIBLE,
  INDEX id_estatus (id_estatus ASC) VISIBLE,
  INDEX idx_fecha (fecha ASC) VISIBLE,
  INDEX idx_ventas_fecha (fecha ASC) VISIBLE,
  INDEX idx_ventas_fecha_metodo (fecha ASC, id_metodo_pago ASC) VISIBLE,
  CONSTRAINT ventas_cabecera_ibfk_1
    FOREIGN KEY (id_metodo_pago)
      REFERENCES tiendita.metodos_pago (id_metodo),
  CONSTRAINT ventas_cabecera_ibfk_2
    FOREIGN KEY (id_estatus)
      REFERENCES tiendita.estatus_venta (id_estatus))
ENGINE = InnoDB
AUTO_INCREMENT = 9
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

```

-- Table tiendita.ventas_detalle

```

DROP TABLE IF EXISTS tiendita.ventas_detalle ;
CREATE TABLE IF NOT EXISTS tiendita.ventas_detalle (
  id_detalle INT NOT NULL AUTO_INCREMENT,
  id_venta INT NOT NULL,
  id_producto INT NOT NULL,
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (id_detalle),
  INDEX idx_venta_producto (id_venta ASC, id_producto ASC) VISIBLE,
  INDEX idx_detalle_producto (id_producto ASC) VISIBLE,
  INDEX idx_detalle_venta (id_venta ASC) VISIBLE,
  CONSTRAINT ventas_detalle_ibfk_1
    FOREIGN KEY (id_venta)
      REFERENCES tiendita.ventas_cabecera (id_venta)

```

```

ON DELETE CASCADE,
CONSTRAINT ventas_detalle_ibfk_2
FOREIGN KEY (id_producto)
REFERENCES tiendita.productos (id_producto))
ENGINE = InnoDB
AUTO_INCREMENT = 9
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

```

```

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

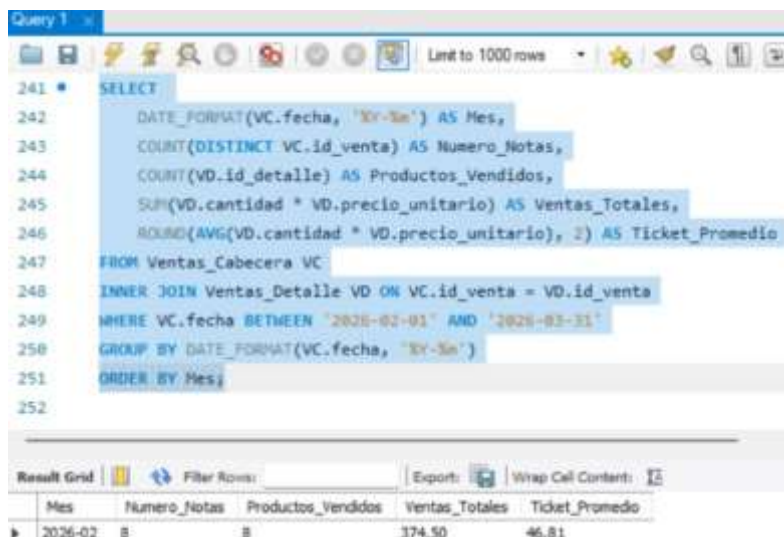
```

1. Ventas del mes de febrero y Marzo

```

SELECT
    DATE_FORMAT(VC.fecha, '%Y-%m') AS Mes,
    COUNT(DISTINCT VC.id_venta) AS Numero_Notas,
    COUNT(VD.id_detalle) AS Productos_Vendidos,
    SUM(VD.cantidad * VD.precio_unitario) AS Ventas_Totales,
    AVG(VD.cantidad * VD.precio_unitario) AS Ticket_Promedio
FROM Ventas_Cabecera VC
INNER JOIN Ventas_Detalle VD ON VC.id_venta = VD.id_venta
WHERE VC.fecha BETWEEN '2026-02-01' AND '2026-03-31'
GROUP BY DATE_FORMAT(VC.fecha, '%Y-%m')
ORDER BY Mes;

```

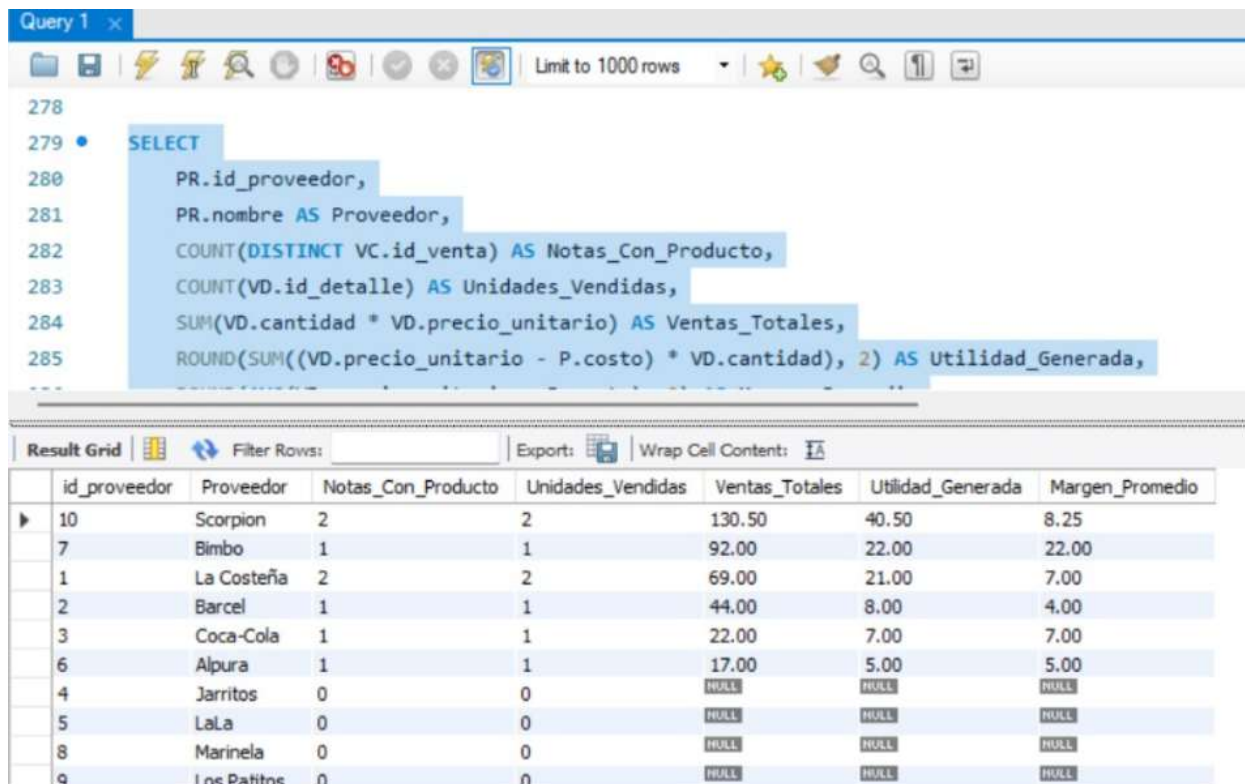


The screenshot shows a SQL query editor with a toolbar at the top. The query is displayed in a text area, and the results are shown in a table below. The table has five columns: Mes, Numero_Notas, Productos_Vendidos, Ventas_Totales, and Ticket_Promedio. The results show data for February 2026.

Mes	Numero_Notas	Productos_Vendidos	Ventas_Totales	Ticket_Promedio
2026-02	8	8	374.50	46.81

2. Ventas por proveedor

```
SELECT
    PR.id_proveedor,
    PR.nombre AS Proveedor,
    COUNT(DISTINCT P.id_producto) AS Productos_Activos,
    SUM(VD.cantidad) AS Unidades_Vendidas,
    SUM(VD.cantidad * VD.precio_unitario) AS Ventas_Totales,
    SUM((VD.precio_unitario - P.costo) * VD.cantidad) AS Utilidad_Total
FROM Proveedores PR
LEFT JOIN Productos P ON PR.id_proveedor = P.id_proveedor
LEFT JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
LEFT JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
GROUP BY PR.id_proveedor, PR.nombre
ORDER BY Ventas_Totales DESC;
```

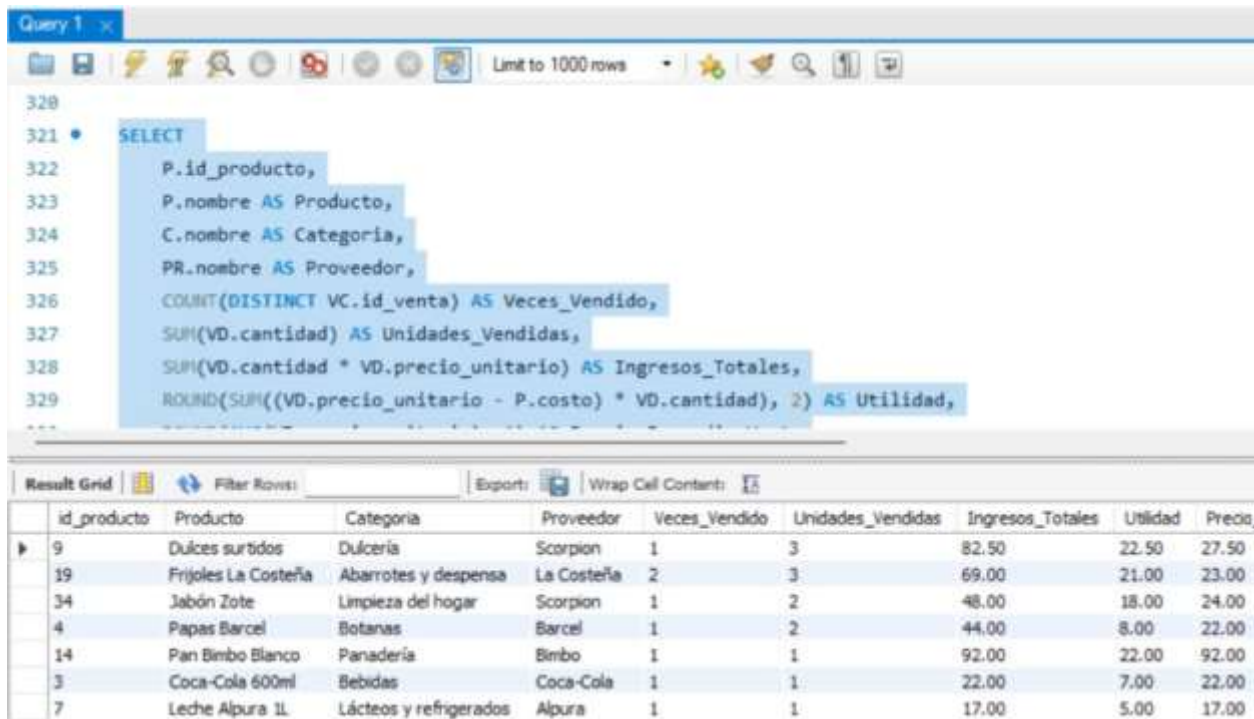


The screenshot shows a database query editor with a toolbar at the top. The query is displayed in the main area, and the results are shown in a grid below. The query is a SELECT statement with several aggregate functions and joins. The results grid has 7 columns: id_proveedor, Proveedor, Notas_Con_Producto, Unidades_Vendidas, Ventas_Totales, Utilidad_Generada, and Margen_Promedio. The data is sorted by Ventas_Totales in descending order.

id_proveedor	Proveedor	Notas_Con_Producto	Unidades_Vendidas	Ventas_Totales	Utilidad_Generada	Margen_Promedio
10	Scorpion	2	2	130.50	40.50	8.25
7	Bimbo	1	1	92.00	22.00	22.00
1	La Costeña	2	2	69.00	21.00	7.00
2	Barcel	1	1	44.00	8.00	4.00
3	Coca-Cola	1	1	22.00	7.00	7.00
6	Alpura	1	1	17.00	5.00	5.00
4	Jarritos	0	0	NULL	NULL	NULL
5	LaLa	0	0	NULL	NULL	NULL
8	Marinela	0	0	NULL	NULL	NULL
9	Los Patitos	0	0	NULL	NULL	NULL

3. Ventas por Producto

```
SELECT
    P.id_producto,
    P.nombre AS Producto,
    C.nombre AS Categoria,
    PR.nombre AS Proveedor,
    COUNT(DISTINCT VC.id_venta) AS Veces_Comprado,
    SUM(VD.cantidad) AS Unidades_Vendidas,
    SUM(VD.cantidad * VD.precio_unitario) AS Ingresos_Totales,
    SUM((VD.precio_unitario - P.costeo) * VD.cantidad) AS Utilidad
FROM Productos P
INNER JOIN Categorías C ON P.id_categoria = C.id_categoria
INNER JOIN Proveedores PR ON P.id_proveedor = PR.id_proveedor
LEFT JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
LEFT JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
GROUP BY P.id_producto, P.nombre, C.nombre, PR.nombre
ORDER BY Ingresos_Totales DESC;
```

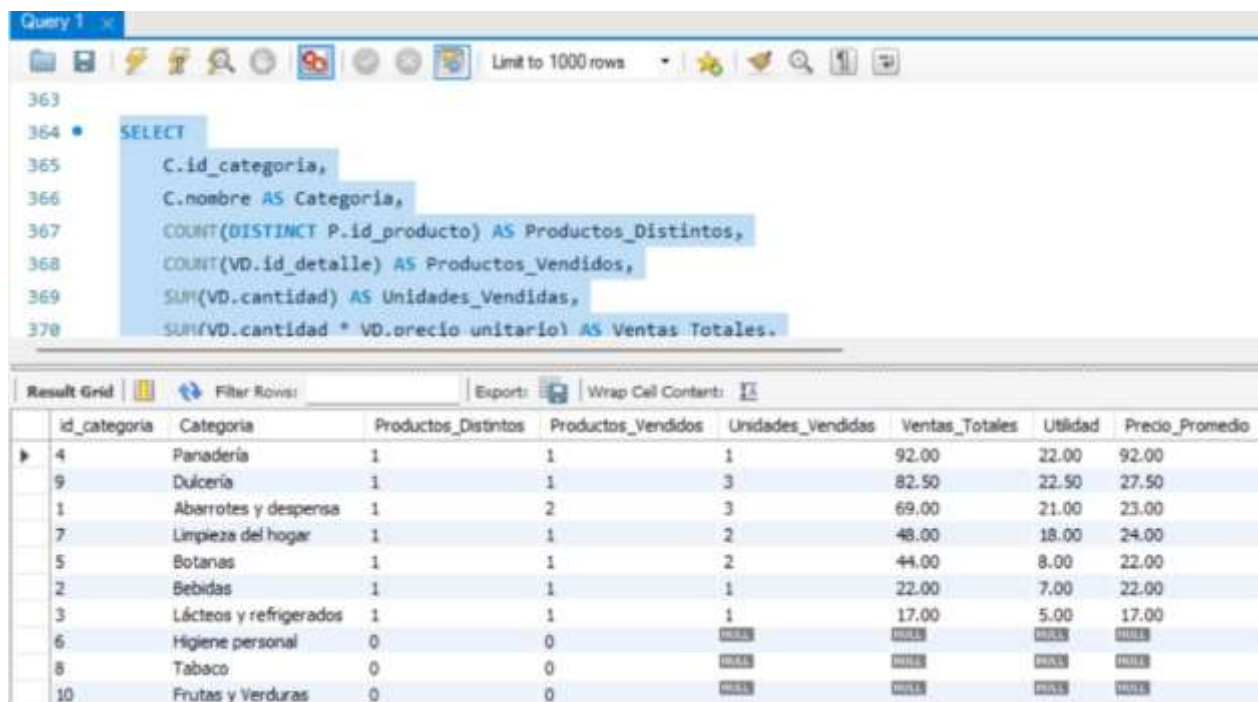


The screenshot shows a SQL query editor window titled "Query 1". The query is a SELECT statement that joins the Productos, Categorías, Proveedores, Ventas_Detalle, and Ventas_Cabecera tables. The query calculates the number of times a product was purchased, the total units sold, the total revenue, and the total profit for each product. The results are ordered by total revenue in descending order.

id_producto	Producto	Categoría	Proveedor	Veces_Vendido	Unidades_Vendidas	Ingresos_Totales	Utilidad	Precio
9	Dulces surtidos	Dulcería	Scorpion	1	3	82.50	22.50	27.50
19	Frijoles La Costeña	Abarrotes y despensa	La Costeña	2	3	69.00	21.00	23.00
34	Jabón Zote	Limpieza del hogar	Scorpion	1	2	48.00	18.00	24.00
4	Papas Barcel	Botanas	Barcel	1	2	44.00	8.00	22.00
14	Pan Bimbo Blanco	Panadería	Bimbo	1	1	92.00	22.00	92.00
3	Coca-Cola 600ml	Bebidas	Coca-Cola	1	1	22.00	7.00	22.00
7	Leche Alpura 1L	Lácteos y refrigerados	Alpura	1	1	17.00	5.00	17.00

4. Ventas por categorías

```
SELECT
    C.id_categoria,
    C.nombre AS Categoria,
    COUNT(DISTINCT P.id_producto) AS Total_Productos,
    SUM(VD.cantidad) AS Unidades_Vendidas,
    SUM(VD.cantidad * VD.precio_unitario) AS Ventas_Totales,
    SUM((VD.precio_unitario - P.costeo) * VD.cantidad) AS Utilidad
FROM Categorías C
LEFT JOIN Productos P ON C.id_categoria = P.id_categoria
LEFT JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
LEFT JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
GROUP BY C.id_categoria, C.nombre
ORDER BY Ventas_Totales DESC;
```



The screenshot shows a database query editor with a toolbar at the top. The query is displayed in a text area, and below it, a 'Result Grid' shows the output. The query is the same as the one provided in the previous block. The result grid has columns for category ID, category name, distinct products, units sold, total sales, utility, and average price. The data is sorted by total sales in descending order.

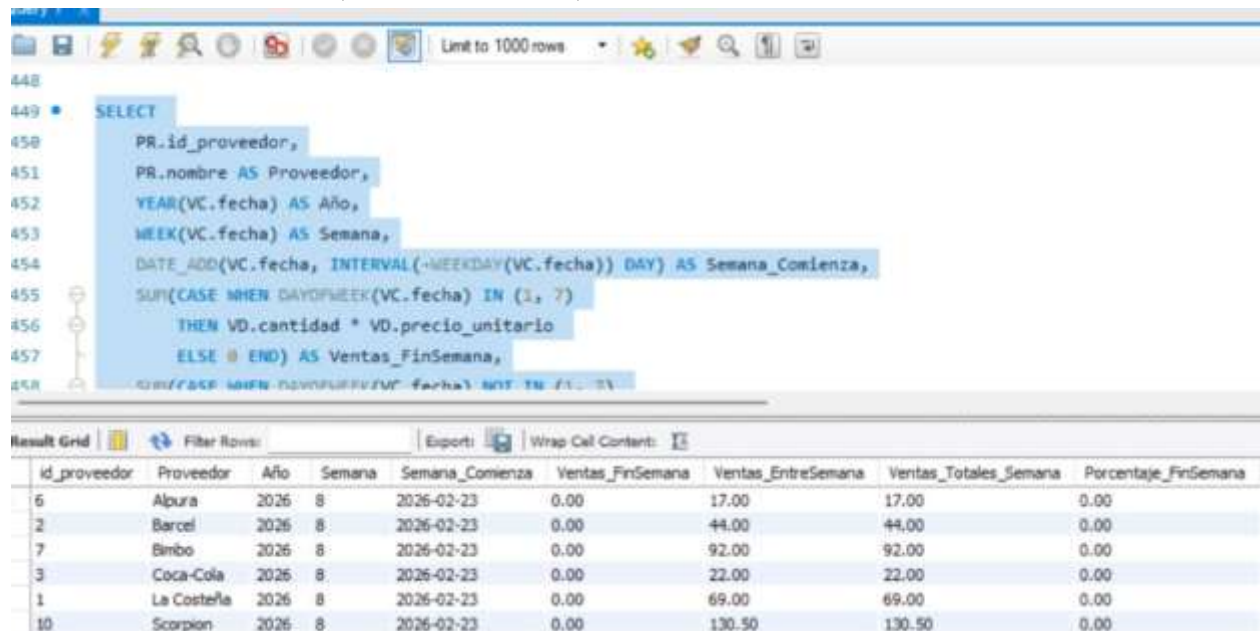
	id_categoria	Categoria	Productos_Distintos	Productos_Vendidos	Unidades_Vendidas	Ventas_Totales	Utilidad	Precio_Promedio
▶	4	Panadería	1	1	1	92.00	22.00	92.00
	9	Dulcería	1	1	3	82.50	22.50	27.50
	1	Abarrotes y despensa	1	2	3	69.00	21.00	23.00
	7	Limpieza del hogar	1	1	2	48.00	18.00	24.00
	5	Botanas	1	1	2	44.00	8.00	22.00
	2	Bebidas	1	1	1	22.00	7.00	22.00
	3	Lácteos y refrigerados	1	1	1	17.00	5.00	17.00
	6	Higiene personal	0	0	NULL	NULL	NULL	NULL
	8	Tabaco	0	0	NULL	NULL	NULL	NULL
	10	Frutas y Verduras	0	0	NULL	NULL	NULL	NULL

5. Ventas por cada fin de semana

```

SELECT
    YEAR(VC.fecha) AS Año,
    WEEK(VC.fecha) AS Semana,
    CONCAT(
        DATE_FORMAT(DATE_ADD(VC.fecha, INTERVAL(-WEEKDAY(VC.fecha)) DAY),
        '%d/%m'),
        '-',
        DATE_FORMAT(DATE_ADD(VC.fecha, INTERVAL(6 - WEEKDAY(VC.fecha))
        DAY), '%d/%m')
    ) AS Periodo_Semana,
    COUNT(DISTINCT CASE WHEN DAYOFWEEK(VC.fecha) IN (1, 7) THEN
    VC.id_venta END) AS Ventas_FinSemana,
    SUM(CASE WHEN DAYOFWEEK(VC.fecha) IN (1, 7)
        THEN VD.cantidad * VD.precio_unitario ELSE 0 END) AS Total_FinSemana,
    SUM(CASE WHEN DAYOFWEEK(VC.fecha) NOT IN (1, 7)
        THEN VD.cantidad * VD.precio_unitario ELSE 0 END) AS Total_EntreSemana
FROM Ventas_Cabecera VC
INNER JOIN Ventas_Detalle VD ON VC.id_venta = VD.id_venta
WHERE VC.fecha BETWEEN '2026-02-01' AND '2026-03-31'
GROUP BY YEAR(VC.fecha), WEEK(VC.fecha), Periodo_Semana
ORDER BY Año DESC, Semana DESC;

```



The screenshot shows a database query editor with the SQL query from the previous block. Below the query, a 'Result Grid' displays the results of the query. The grid has 9 columns: id_proveedor, Proveedor, Año, Semana, Semana_Comienzo, Ventas_FinSemana, Ventas_EntreSemana, Ventas_Totales_Semana, and Porcentaje_FinSemana. The results show data for 6 different providers (Alpura, Barcel, Bimbo, Coca-Cola, La Costeña, Scorpion) for the year 2026, week 8, starting on 2026-02-23. The 'Ventas_FinSemana' column shows 0.00 for all providers, while 'Ventas_EntreSemana' and 'Ventas_Totales_Semana' show values for each provider.

id_proveedor	Proveedor	Año	Semana	Semana_Comienzo	Ventas_FinSemana	Ventas_EntreSemana	Ventas_Totales_Semana	Porcentaje_FinSemana
6	Alpura	2026	8	2026-02-23	0.00	17.00	17.00	0.00
2	Barcel	2026	8	2026-02-23	0.00	44.00	44.00	0.00
7	Bimbo	2026	8	2026-02-23	0.00	92.00	92.00	0.00
3	Coca-Cola	2026	8	2026-02-23	0.00	22.00	22.00	0.00
1	La Costeña	2026	8	2026-02-23	0.00	69.00	69.00	0.00
10	Scorpion	2026	8	2026-02-23	0.00	130.50	130.50	0.00

6. Ventas del proveedor en cada fin de semana (febrero / Marzo)

```
SELECT
    PR.id_proveedor,
    PR.nombre AS Proveedor,
    YEAR(VC.fecha) AS Año,
    WEEK(VC.fecha) AS Semana,
    CONCAT(
        DATE_FORMAT(DATE_ADD(VC.fecha, INTERVAL(-WEEKDAY(VC.fecha)) DAY),
'%d/%m'),
        '-',
        DATE_FORMAT(DATE_ADD(VC.fecha, INTERVAL(6 - WEEKDAY(VC.fecha))
DAY), '%d/%m')
    ) AS Periodo_Semana,
    SUM(CASE WHEN DAYOFWEEK(VC.fecha) IN (1, 7)
        THEN VD.cantidad * VD.precio_unitario ELSE 0 END) AS Ventas_FinSemana,
    SUM(CASE WHEN DAYOFWEEK(VC.fecha) NOT IN (1, 7)
        THEN VD.cantidad * VD.precio_unitario ELSE 0 END) AS Ventas_EntreSemana,
    SUM(VD.cantidad * VD.precio_unitario) AS Ventas_Totales_Semana
FROM Proveedores PR
INNER JOIN Productos P ON PR.id_proveedor = P.id_proveedor
INNER JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
INNER JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
WHERE VC.fecha BETWEEN '2026-02-01' AND '2026-03-31'
GROUP BY PR.id_proveedor, PR.nombre, YEAR(VC.fecha), WEEK(VC.fecha),
Periodo_Semana
```

The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 504 to 508. The query is a SELECT statement filtering for provider ID 10. The bottom pane shows the 'Result Grid' with a table containing three columns: id_proveedor, nombre, and fecha_registro. The first row shows the results for provider 10, and the second row shows NULL values.

```
504 -- PRIMERO: Verificar quién es el proveedor #10
505 • SELECT * FROM Proveedores WHERE id_proveedor = 10;
506
507 -- CONSULTA PRINCIPAL
508 • SELECT
```

	id_proveedor	nombre	fecha_registro
▶	10	Scorpion	2026-03-08 09:52:25
•	NULL	NULL	NULL

7. Ventas del proveedor No. 10 en el mes de febrero

```
SELECT
    PR.nombre AS Proveedor,
    P.id_producto,
    P.nombre AS Producto,
    C.nombre AS Categoria,
    COUNT(DISTINCT VC.id_venta) AS Veces_Vendido,
    SUM(VD.cantidad) AS Unidades_Vendidas,
    AVG(VD.precio_unitario) AS Precio_Promedio,
    SUM(VD.cantidad * VD.precio_unitario) AS Ingresos_Totales,
    SUM((VD.precio_unitario - P.costos) * VD.cantidad) AS Utilidad
FROM Proveedores PR
INNER JOIN Productos P ON PR.id_proveedor = P.id_proveedor
INNER JOIN Categorías C ON P.id_categoria = C.id_categoria
LEFT JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
LEFT JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
    AND VC.fecha BETWEEN '2026-02-01' AND '2026-02-28'
WHERE PR.id_proveedor = 10
GROUP BY PR.nombre, P.id_producto, P.nombre, C.nombre
ORDER BY Ingresos_Totales DESC;
```

-- Versión resumen (totales del proveedor)

```
SELECT
    PR.nombre AS Proveedor,
    COUNT(DISTINCT P.id_producto) AS Productos_Activos,
    SUM(VD.cantidad) AS Unidades_Vendidas,
    SUM(VD.cantidad * VD.precio_unitario) AS Ingresos_Totales,
    SUM((VD.precio_unitario - P.costos) * VD.cantidad) AS Utilidad_Total
FROM Proveedores PR
INNER JOIN Productos P ON PR.id_proveedor = P.id_proveedor
LEFT JOIN Ventas_Detalle VD ON P.id_producto = VD.id_producto
LEFT JOIN Ventas_Cabecera VC ON VD.id_venta = VC.id_venta
    AND VC.fecha BETWEEN '2026-02-01' AND '2026-02-28'
WHERE PR.id_proveedor = 10
GROUP BY PR.id_proveedor, PR.nombre;
```

Query 1SQL File 5

Limit to 1000 rows